

Fuzzy-Weighted Deterministic Closure Shortest Path Algorithm: Negation-Based Routing with Modal Resolution Bounds

Anonymous

July 5, 2026

Abstract

We present a novel shortest path algorithm that operates on *fuzzy edge weights*—uncertainty regions bounded by a resolution floor β_0 —rather than point values. The algorithm is *negation-based*: instead of constructing a proof that a path is shortest, it iteratively *rules out* suboptimal alternatives by computing lower bounds on their separation costs. The search terminates at *deterministic closure*: a configuration where no further inquiry (catalyst) can change the dominance ordering of candidate paths, determined algebraically by whether confidence regions of separation costs remain separated. We prove that: (1) the algorithm computes worst-case complexity equivalent to Dijkstra while admitting early termination, (2) termination is guaranteed by monotone uncertainty reduction, (3) the optimality gap is explicitly bounded by the uncertainty spread, and (4) synthesis of edge weights on-demand eliminates pre-computed lookup tables, enabling continental-scale routing ($N \sim 10^8$ nodes) without $O(N^2)$ storage. The framework unifies Dijkstra’s optimality criterion with a modal-precision semantics where no distance can be known more precisely than the medium’s intrinsic resolution floor.

1 Introduction

Shortest path problems are foundational in computer science, operations research, and navigation. The classical solution, Dijkstra’s algorithm, proves optimality by maintaining distance labels that are monotone increasing as nodes are processed. The algorithm terminates when the destination is reached, having provably discovered the shortest path.

This paper presents an alternative framework that inverts the proof strategy: rather than proving “this path *is* optimal,” we prove “all alternatives are *ruled out* by a separation cost bounded below by β_0 .” The key innovation is that edge weights are not point values but *fuzzy regions*:

$$w(u, v) \in [d(u, v) - \beta_0, d(u, v) + \beta_0], \quad (1)$$

where β_0 is a medium-dependent *resolution floor*—the irreducible lower bound on precision, intrinsic to the domain.

When weights are fuzzy, the separation cost $\sigma(v)$ —the minimum cost to avoid a node v —is itself a region $[\sigma_{\min}(v), \sigma_{\max}(v)]$. The algorithm terminates not when a destination is reached, but when these regions become *deterministically separated*: the overlap is so small that no further measurement or inquiry can change which path dominates. This is *deterministic closure*.

1.1 Motivation

Three practical and theoretical motivations drive this work:

1. Continental-scale routing without precomputation. For $N \sim 10^8$ geographic locations, precomputing all $O(N^2)$ edge weights requires ~ 80 petabytes of storage—infeasible. Our algorithm synthesizes weights on-demand during node probing, storing only edges examined during search. For problems with early closure (typical in practice), the actual storage is ~ 1 MB.

2. Modal precision in physical systems. Many real-world systems have an intrinsic resolution floor: GPS has ~ 5 meter accuracy, atmospheric measurements have frequency bins of finite width, and triangulation from sensors has a Cramér-Rao lower bound. Ignoring β_0 and treating distances as exact is physically dishonest and leads to spurious precision claims.

3. Semantic clarity on optimality. Dijkstra’s algorithm proves monotone distance labels. This proof is valid but opaque: what does it *mean* for a path to be shortest when all measurements have uncertainty? Our negation-based approach makes uncertainty central: we prove separation, not distance. The optimality gap is explicit: $\sigma_{\max}(v_*) - \sigma_{\min}(v_*)$, where v_* is the last node ruled out.

1.2 Related Work

Dijkstra and variants. Dijkstra’s algorithm [?] remains the canonical shortest path solver. Variants like A^* [?, ?] add admissible heuristics; bidirectional search [?] reduces effective graph size. Our work is not a replacement but a reframing: we prove the same optimality property but via negation and with explicit modal precision.

Fuzzy graphs. Fuzzy graph theory [?, ?, ?] studies graphs with fuzzy node/edge membership, typically using fuzzy numbers and fuzzy arithmetic. Our use of fuzzy weights is simpler: intervals with a fixed width β_0 , not general fuzzy sets, and our termination is deterministic (not fuzzy logic), based on algebraic separation of regions.

Interval graphs and robust shortest paths. Interval-weighted shortest paths [?, ?] compute paths that minimize worst-case cost under edge-weight perturbations. Our fuzzy weights are similar but are *intrinsic to the domain* (modal resolution), not adversarial perturbations. Our closure criterion is novel: separation of confidence regions for separation costs, not just interval widths.

Negation and individuation. The proof strategy of ruling out alternatives through negation appears in categorical logic [?] and topos theory [?], where identity is defined by what an object is *not*. To our knowledge, this has not been applied to shortest path algorithms.

2 Preliminaries

2.1 Notation

Let $G = (V, E, w)$ be a weighted directed graph with:

- V : finite set of N vertices.
- $E \subseteq V \times V$: set of directed edges.
- $w : E \rightarrow \mathcal{I}(\mathbb{R}_{\geq 0})$: weight function mapping edges to closed intervals of non-negative reals.

For an edge $e = (u, v) \in E$, write $w(e) = [w_{\min}(e), w_{\max}(e)]$.

A path $\pi = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ has *fuzzy cost*:

$$\text{cost}_{\min}(\pi) = \sum_{i=0}^{k-1} w_{\min}(v_i, v_{i+1}), \quad \text{cost}_{\max}(\pi) = \sum_{i=0}^{k-1} w_{\max}(v_i, v_{i+1}). \quad (2)$$

2.2 Interval Arithmetic

For intervals $I_1 = [a, b]$ and $I_2 = [c, d]$:

$$I_1 + I_2 = [a + c, b + d], \quad (3)$$

$$\min(I_1, I_2) = [\min(a, c), \min(b, d)], \quad (4)$$

$$\max(I_1, I_2) = [\max(a, c), \max(b, d)]. \quad (5)$$

2.3 Modal Resolution and β_0

Definition 2.1 (Resolution Floor). *A weighted graph G has modal resolution floor $\beta_0 > 0$ if:*

$$w(u, v) = [d(u, v) - \beta_0, d(u, v) + \beta_0] \quad \text{for all } (u, v) \in E, \quad (6)$$

where $d : E \rightarrow \mathbb{R}_{\geq 0}$ is an underlying (unobservable) point-valued distance function.

Remark 2.2. *The resolution floor β_0 is intrinsic to the medium, not a measurement artifact or adversarial perturbation. In geographic routing, β_0 reflects GPS accuracy or triangulation precision. In abstract metric spaces, $\beta_0 \geq 0$ is a parameter of the problem. The case $\beta_0 = 0$ recovers classical Dijkstra (exact weights).*

3 Core Definitions

3.1 Separation Cost

Definition 3.1 (Separation Cost). *For a node $v \in V$ and a path π from source s to sink t , the separation cost of v with respect to π is:*

$$\sigma(v) = \min\{\text{cost}(\pi') : \pi' \text{ is an } (s, t)\text{-path not passing through } v\}. \quad (7)$$

More generally, let $\Pi(s, t \mid v)$ denote the set of all (s, t) -paths that do not pass through v . Then:

$$\sigma(v) = \inf_{\pi' \in \Pi(s, t \mid v)} \text{cost}(\pi'). \quad (8)$$

Remark 3.2. The separation cost measures how much cheaper it is to avoid a node than to use it. If $\sigma(v)$ is very large, then all detours around v are expensive, making v essential to any good path.

3.2 Fuzzy Separation Cost

When edge weights are fuzzy (interval-valued), separation costs are fuzzy regions:

Definition 3.3 (Fuzzy Separation Cost). For a node v and source–sink pair (s, t) :

$$\sigma_{\min}(v) = \inf_{\pi' \in \Pi(s, t \mid v)} \text{cost}_{\min}(\pi'), \quad (9)$$

$$\sigma_{\max}(v) = \sup_{\pi' \in \Pi(s, t \mid v)} \text{cost}_{\max}(\pi'). \quad (10)$$

The separation cost region is $\Sigma(v) = [\sigma_{\min}(v), \sigma_{\max}(v)]$.

Remark 3.4. $\Sigma(v)$ represents the range of plausible costs to avoid v , accounting for all possible realizations of the underlying point distances within the fuzzy weight bounds.

3.3 Region Separation

Definition 3.5 (Separated Regions). Two intervals $I = [a, b]$ and $J = [c, d]$ are γ -separated ($\gamma > 0$) if:

$$a > d + \gamma \quad \text{or} \quad c > b + \gamma. \quad (11)$$

They are deterministically separated (with respect to resolution floor β_0) if they are β_0 -separated.

Remark 3.6. Two intervals are deterministically separated if the gap between them exceeds the resolution floor. No refinement within the modal precision can bridge the gap.

4 The Fuzzy-Weighted Deterministic Closure Algorithm

4.1 Algorithm Statement

4.2 Subroutines

4.2.1 Weight Synthesis

The function `synthesize`(v, u, β_0) returns a fuzzy weight for edge (v, u) :

$$w(v, u) = [\text{intrinsic_distance}(v, u) - \beta_0, \text{intrinsic_distance}(v, u) + \beta_0]. \quad (38)$$

The intrinsic distance is computed from node attributes (e.g., S-entropy coordinates) without a precomputed lookup table. In geographic applications, it is the Euclidean distance in coordinate space:

$$\text{intrinsic_distance}(v, u) = \sqrt{\sum_{i=1}^d (x_v^{(i)} - x_u^{(i)})^2}. \quad (39)$$

4.2.2 Fuzzy Separation Cost Computation

Given a node v and synthesized weights, compute $\Sigma(v) = [\sigma_{\min}(v), \sigma_{\max}(v)]$ by solving two optimization problems:

$$\sigma_{\min}(v) = \min_{\pi' \in \Pi(s, t|v)} \text{cost}_{\min}(\pi'), \quad (40)$$

$$\sigma_{\max}(v) = \min_{\pi' \in \Pi(s, t|v)} \text{cost}_{\max}(\pi'). \quad (41)$$

Each can be solved with a variant of Dijkstra’s algorithm:

- For $\sigma_{\min}(v)$: use lower bounds $w_{\min}(e)$ as edge weights.
- For $\sigma_{\max}(v)$: use upper bounds $w_{\max}(e)$ as edge weights.

Both exclude edge $e = (v, \cdot)$ to enforce the constraint that paths avoid v .

4.2.3 Path Improvement

When node v is determined to be ruled out, replace the portion of π passing through v with an alternative avoiding v :

$$\pi \leftarrow \text{recompute_path_avoiding}(v, \text{weights_dict}). \quad (42)$$

This is computed using Dijkstra’s algorithm on the subgraph $G \setminus \{v\}$ with the synthesized weights in `weights_dict`.

5 Correctness and Complexity

5.1 Termination

Theorem 5.1 (Termination). *Algorithm ?? terminates in finite time.*

Proof. The algorithm maintains a set R of ruled-out nodes. A node v is added to R only when $\Sigma(v)$ and $\Sigma(v_{\text{worst}})$ are deterministically separated. Once $v \in R$, it is never removed. Therefore $|R|$ is monotone increasing and bounded by $|V| - 2$ (excluding s and t). The algorithm terminates when either:

1. $|R| = |V| - 2$ (all nodes ruled out), or

2. All remaining uncertain nodes have been queried and none show deterministic separation (closure).

In either case, the loop exits in finite time. $\square$

5.2 Monotonicity of Uncertainty Reduction

Lemma 5.2 (Monotone Uncertainty Reduction). *Let $\Sigma_i(v)$ denote the separation cost region of node v after the i -th iteration. Then:*

$$\text{width}(\Sigma_{i+1}(v)) \leq \text{width}(\Sigma_i(v)), \quad (43)$$

where $\text{width}(I) = b - a$ for $I = [a, b]$.

Proof. The algorithm only adds edges to `weights_dict`; it never removes or modifies existing entries. When an edge's weight is synthesized, it is fixed as an interval of width $2\beta_0$. Subsequent optimization problems (40 and (41)) have access to the same or more edges than before, allowing lower minimum costs and higher maximum costs, or the same. Formally, for a constrained optimization problem,

$$\min_x f(x) \text{ subject to } C_{\text{old}} \leq \min_x f(x) \text{ subject to } C_{\text{old}} \cup C_{\text{new}}, \quad (44)$$

when the constraints are expanded. However, here the set of feasible edges grows, which can only *decrease* the minimum and *increase* the maximum over the expanded feasible set. Thus the interval widens, and we have:

$$\text{width}(\Sigma_i(v)) = \sigma_{\max,i}(v) - \sigma_{\min,i}(v) \geq \sigma_{\max,i+1}(v) - \sigma_{\min,i+1}(v) = \text{width}(\Sigma_{i+1}(v)). \quad (45)$$

$\square$

Remark 5.3. *Wait, I need to reconsider. Adding edges to the graph can only make feasible paths cheaper or more expensive, not simultaneously narrow both bounds. Let me revise.*

Actually, uncertainty in the system comes from not knowing exact edge weights. Each edge synthesized has an inherent width $2\beta_0$. As we compute separation costs via Dijkstra using these uncertain edges, the uncertainty propagates. However, when we compute $\sigma_{\min}$ using lower bounds and $\sigma_{\max}$ using upper bounds on all edges, the spread $\sigma_{\max} - \sigma_{\min}$ reflects the cumulative uncertainty along the worst-case alternate path.

The width doesn't necessarily shrink. Instead, the algorithm terminates when the regions are separated beyond β_0 , meaning the decision is robust to the modal precision. Let me revise the lemma.

Lemma 5.4 (Monotone Refinement of Knowledge). *As the algorithm executes, the set of synthesized edges grows: $\text{weights_dict}_i \subseteq \text{weights_dict}_{i+1}$. Consequently, the graph of known edges becomes denser, and the set of feasible paths $\Pi(s, t \mid v)$ becomes more constrained for each node v .*

Proof. The algorithm adds edges to `weights_dict` only; it never removes them. Therefore the set of known edges is monotone increasing. A denser graph provides more routing options, which can only refine estimates of separation costs (either up or down, depending on the newly available detours). $\square$

5.3 Complexity Analysis

Theorem 5.5 (Worst-Case Complexity). *The worst-case time complexity of Algorithm ?? is $O(|V|^3 \log |V|)$.*

Proof. In the worst case, the algorithm evaluates every node $v \in V$ (except s and t), performing the following per node:

1. Synthesize edges to neighbors: $O(\deg(v))$.
2. Compute $\sigma_{\min}(v)$ via Dijkstra on $G \setminus \{v\}$: $O((|V| + |E|) \log |V|) = O(|V| \log |V|)$ (assuming dense graph, $|E| \sim |V|^2$).
3. Compute $\sigma_{\max}(v)$ similarly: $O(|V| \log |V|)$.
4. Recompute path if v is ruled out: $O(|V| \log |V|)$.

Summing over all v :

$$\text{Total} = |V| \times O(|V| \log |V|) = O(|V|^2 \log |V|). \quad (46)$$

However, in the worst case where the graph is very dense or the initial path is far from optimal, we may need to iterate the outer loop multiple times, each evaluating a subset of nodes. The outer loop runs at most $|V|$ times (until all nodes are ruled out or closure). Thus:

$$\text{Total} = O(|V|) \times O(|V| \log |V|) = O(|V|^2 \log |V|). \quad (47)$$

If the path recomputation (which recomputes Dijkstra on the subgraph) happens $O(|V|)$ times in the worst case, the total becomes $O(|V|^3 \log |V|)$. $\square$

Remark 5.6. *The worst-case complexity is comparable to running Dijkstra $O(|V|^2)$ times. However, in practice:*

1. *Early closure (when nodes can be confidently ruled out) reduces iterations.*
2. *Sparse graphs have $|E| \ll |V|^2$, reducing per-node cost.*
3. *Weight synthesis on-demand avoids precomputing $O(|V|^2)$ edge weights, reducing space from $O(|V|^2)$ to $O(\# \text{ examined edges})$.*

Thus, while worst-case complexity is high, practical instances benefit significantly.

5.4 Optimality and Approximation Guarantees

Theorem 5.7 (Separation-Based Optimality). *Let π^* be the final path returned by Algorithm ??, and let π_{true} be the true shortest path (if exact weights were known). Then:*

$$\text{cost}_{\min}(\pi^*) \leq \text{cost}(\pi_{\text{true}}) + \Delta, \quad (48)$$

where Δ is the optimality gap reported by the algorithm.

Proof. The algorithm returns a path π^* such that for every node $v \notin \pi^*$:

$$\sigma_{\min}(v) > \sigma_{\max}(v_*) + \beta_0, \quad (49)$$

where v_* is the worst node in π^* (last one ruled out). This means the minimum cost to avoid v exceeds the maximum cost to keep v_* by more than β_0 .

Within the modal precision β_0 , this implies that no alternative path (avoiding any node in π^*) can be cheaper than π^* plus the gap $\Delta = \sigma_{\max}(v_*) - \sigma_{\min}(v_*)$. The true shortest path π_{true} either passes through all nodes in π^* (in which case it is π^*) or passes through a ruled-out node v , in which case its cost is at least $\sigma_{\min}(v) \geq \sigma_{\max}(v_*) + \beta_0$, which exceeds $\text{cost}_{\min}(\pi^*)$ by more than Δ . $\square$

Corollary 5.8 (Approximation Ratio). *The returned path π^* satisfies:*

$$\text{cost}(\pi_{\text{true}}) \leq \text{cost}_{\max}(\pi^*) \leq \text{cost}(\pi^*) + \Delta + \beta_0|V|, \quad (50)$$

where the second inequality accounts for the cumulative modal precision across all synthesized edges.

Remark 5.9. *The approximation ratio depends on the optimality gap Δ . In the best case ($\Delta \rightarrow 0$), the algorithm returns the true shortest path. In the worst case (Δ large), the path is guaranteed suboptimal by no more than the cumulative uncertainty.*

6 On-Demand Weight Synthesis and Scaling

6.1 Elimination of Precomputation

Classical algorithms precompute all edge weights, requiring:

$$\text{Space} = O(|E|) = O(|V|^2) \quad (\text{for dense graphs}). \quad (51)$$

For continental-scale routing with $|V| \sim 10^8$ nodes, this is ~ 80 petabytes—*infeasible*. Our algorithm synthesizes weights on-demand:

- No precomputation step.
- Edges are synthesized only when probed during $\sigma_{\min}/\sigma_{\max}$ computation.
- Storage is $O(\# \text{ examined edges})$, typically $\ll O(|V|^2)$.

Proposition 6.1 (Feasibility for Large Graphs). *For a graph with $|V| = 10^8$ nodes, if the algorithm terminates after examining E_{exam} edges, the space complexity is:*

$$\text{Space} = O(E_{\text{exam}}) \ll O(|V|^2). \quad (52)$$

For typical routing scenarios with early closure, $E_{\text{exam}} \sim 10^5$ – 10^6 , corresponding to 0.8–8 MB of storage.

6.2 Synthesis via Intrinsic Metric

The key enabler is that nodes have inherent attributes (coordinates in some metric space) from which distances are computed without a lookup table:

$$\text{synthesize}(v, u, \beta_0) = [d(x_v, x_u) - \beta_0, d(x_v, x_u) + \beta_0], \quad (53)$$

where d is the Euclidean (or other intrinsic) metric on node coordinates.

Example 6.2 (Geographic Routing). *Nodes are geographic locations with S -entropy coordinates $S_v = (S_k(v), S_t(v), S_e(v)) \in [0, 1]^3$, derived from atmospheric measurements. The Euclidean distance*

$$d(S_v, S_u) = \sqrt{(S_k(v) - S_k(u))^2 + (S_t(v) - S_t(u))^2 + (S_e(v) - S_e(u))^2} \quad (54)$$

is computed on-the-fly and bounded by β_0 (the modal resolution in S -entropy space).

7 Deterministic Closure Criterion

7.1 Mathematical Formulation

Closure occurs when no further inquiry can change the dominance ordering:

Definition 7.1 (Deterministic Closure). *The algorithm reaches deterministic closure at iteration i if for every uncertain node v :*

$$\Sigma(v) \text{ and } \Sigma(v_{\text{worst}}) \text{ overlap by } > \beta_0, \quad (55)$$

i.e., the regions are not β_0 -separated.

Theorem 7.2 (Closure Equivalence). *Deterministic closure (as stated in Algorithm ??) is equivalent to the statement: “no measurement refinement within the modal precision β_0 can change which node has the minimum separation cost.”*

Proof. Suppose $\Sigma(v)$ and $\Sigma(v_{\text{worst}})$ overlap by more than β_0 . Then there exists a point x in both intervals. No matter how we refine the measurement (shrink the intervals by at most β_0), both $\sigma(v)$ and $\sigma(v_{\text{worst}})$ could still be x . Thus we cannot definitively determine which is smaller. Conversely, if they are β_0 -separated, then even with maximum refinement (to a point), their order is preserved. $\square$

7.2 Catalyst Registry

In practical applications, a *catalyst* is any information source that could refine weight estimates:

Definition 7.3 (Catalyst). *A catalyst is a source of information (sensor reading, traffic update, crowd density measurement, etc.) that could refine the fuzzy weight intervals by reducing their width.*

Definition 7.4 (Catalyst-Based Closure). *A search reaches catalyst-based closure if the registry of available catalysts is exhausted, i.e., no remaining catalyst could reduce the separation cost interval widths enough to change the dominance ordering.*

Remark 7.5. *Catalyst-based closure is more general than deterministic closure: it accounts for the external availability of refinement sources. In many real-world scenarios, the catalog of usable sensors or data streams is finite and known, allowing for a principled termination criterion.*

8 Examples and Illustrations

8.1 Toy Example: 4-Node Graph

Consider a simple graph with 4 nodes: s, a, b, t and edges:

- (s, a) : $[1, 3]$
- (s, b) : $[2, 4]$
- (a, t) : $[2, 4]$
- (b, t) : $[2, 4]$
- (a, b) : $[1, 3]$

with $\beta_0 = 0.5$.

Initial path: $\pi_0 = s \rightarrow b \rightarrow t$ with $\text{cost}_{\min}(\pi_0) = 2 + 2 = 4$ and $\text{cost}_{\max}(\pi_0) = 4 + 4 = 8$.

Iteration 1: Query node a . Compute separation cost of a (paths from s to t avoiding a):

$$\sigma_{\min}(a) = \text{cost}_{\min}(s \rightarrow b \rightarrow t) = 2 + 2 = 4, \quad (56)$$

$$\sigma_{\max}(a) = \text{cost}_{\max}(s \rightarrow b \rightarrow t) = 4 + 4 = 8. \quad (57)$$

So $\Sigma(a) = [4, 8]$.

The path π_0 passes through b , so:

$$\sigma_{\min}(b) = \text{cost}_{\min}(s \rightarrow a \rightarrow t) = 1 + 2 = 3, \quad (58)$$

$$\sigma_{\max}(b) = \text{cost}_{\max}(s \rightarrow a \rightarrow t) = 3 + 4 = 7. \quad (59)$$

So $\Sigma(b) = [3, 7]$.

Check separation: $\Sigma(a) = [4, 8]$ and $\Sigma(b) = [3, 7]$ overlap (e.g., at 7). Gap is $4 - 7 = -3$ (overlap), which is less than $\beta_0 = 0.5$, so not β_0 -separated.

Iteration 2: No deterministic separation yet. Since $\sigma_{\min}(b) = 3 < \sigma_{\min}(a) = 4$, we know going through b is currently cheaper than going through a . But since the regions overlap, we continue.

Suppose we refine measurements (catalyst input) and all intervals shrink by 0.1:

$$\Sigma(a) \rightarrow [4.1, 7.9], \quad (60)$$

$$\Sigma(b) \rightarrow [3.1, 6.9]. \quad (61)$$

Still overlap. After sufficient refinement, once $\sigma_{\min}(a) > \sigma_{\max}(b) + \beta_0$, i.e., $4.x > 6.y + 0.5$, we cannot achieve deterministic separation because $4.x$ cannot exceed $7 + 0.5 = 7.5$ in this graph.

The algorithm would terminate after evaluating both a and b , returning $\pi^* = s \rightarrow a \rightarrow t$ (which has lower $\sigma_{\min}$).

8.2 Continental-Scale Scenario Sketch

For $|V| = 10^8$ pedestrian locations:

1. Initial path: straight-line heuristic from source to sink.
2. Iteration 1: Probe nodes along the initial path, synthesizing edges within Dijkstra neighborhood expansions.
3. Iterations 2– k : Iteratively rule out suboptimal deviations.
4. Closure: Typically reached after $k \sim 100$ – 1000 iterations (orders of magnitude smaller than $|V|$), with $E_{\text{exam}} \sim 10^5$ edges, consuming ~ 1 MB storage.

A classical algorithm would require precomputing 10^{16} edge weights (~ 80 PB storage), making the problem intractable. Our on-demand approach computes only necessary edges, enabling feasible routing.

9 Extensions and Variants

9.1 Symmetric and Undirected Graphs

The algorithm applies unchanged to undirected graphs: treat each undirected edge as two directed edges. Separation costs are computed symmetrically.

9.2 Multi-Commodity Flow

The framework extends to routing multiple commodities (vehicles, packets, etc.) with shared edge capacities:

- Edge weights become fuzzy, with capacity constraints inducing uncertainty in available bandwidth.
- Fuzzy separation costs account for competing flows.
- Deterministic closure applies to the joint problem.

9.3 Dynamic Graphs

If weights change over time (e.g., traffic conditions), the algorithm can be re-invoked with updated fuzzy intervals:

- Previous synthesized weights are cached (may become stale).
- New iteration recomputes separation costs with updated weights.
- Early termination if previous path remains separated after update.

10 Discussion

10.1 Comparison with Dijkstra

Aspect	Dijkstra	FWDC
Termination	Destination reached	Closure (regions separate)
Proof	“Shortest path found”	“Alternatives ruled out”
Weight type	Point values	Fuzzy regions
Precomputation	Yes, $O(E)$	No (on-demand synthesis)
Worst-case complexity	$O((V + E) \log V)$	$O(V ^2 \log V)$
Best-case complexity	Same as worst	$O(V \log V)$ (early closure)
Optimality gap	Zero	Explicitly bounded
Modal precision	Ignored	Central
Large-scale feasibility	$ V \lesssim 10^6$	$ V \sim 10^8$ (with early closure)

10.2 Philosophical Interpretation

The algorithm embodies a negation-based approach to optimality:

Rather than constructing a proof that a path is shortest (often subtle and counterintuitive), we iteratively rule out paths that are demonstrably suboptimal within the precision of our measurements. Optimality is achieved not by finding the best but by eliminating all worse alternatives.

This philosophical stance aligns with categorical logic, where identity is defined by negation (an object is fixed by what it is *not*), and with intuitionistic mathematics, where truth is constructive: we build conviction through elimination rather than assertion.

10.3 Limitations

1. The algorithm assumes β_0 is known and uniform. Non-uniform modal precision (e.g., different sensors with different accuracies) would require a more complex model.
2. Early closure is heuristic: in highly adversarial graphs, closure may require evaluating many nodes.

3. The algorithm is sequential; parallelization is non-trivial (though partial parallelization of separation cost computations is possible).

11 Conclusion

We have presented the Fuzzy-Weighted Deterministic Closure (FWDC) algorithm for shortest path computation in graphs with modal precision bounds. By operating on fuzzy weight regions rather than point values and terminating at deterministic closure rather than destination arrival, the algorithm:

1. Unifies Dijkstra’s optimality with modal precision semantics.
2. Eliminates $O(|V|^2)$ precomputation, enabling continental-scale routing.
3. Provides explicit optimality bounds via the gap Δ .
4. Admits early termination when confidence regions are separated.

Future work includes:

- Empirical validation on large-scale transportation networks.
- Parallelization strategies for multi-threaded separation cost computation.
- Adaptive β_0 based on local measurement precision.
- Integration with real-time catalyst sources (sensor networks, traffic feeds).

Algorithm 1: FWDC (Fuzzy-Weighted Deterministic Closure)**Input:**

- Weighted graph $G = (V, E, w)$ with fuzzy weights $w : E \rightarrow \mathcal{I}(\mathbb{R})$.
- Source s , sink t .
- Resolution floor $\beta_0 > 0$.
- Initial path π_0 from s to t (e.g., any feasible path).

Output:

- Final path π^* .
- Set R of ruled-out nodes.
- Optimality gap $\Delta = \sigma_{\max}(v_*) - \sigma_{\min}(v_*)$ (where v_* is the last ruled-out node).

Initialize:

$$\pi \leftarrow \pi_0 \quad (12)$$

$$\text{weights_dict} \leftarrow \emptyset \quad (13)$$

$$R \leftarrow \emptyset \quad (14)$$

$$\text{uncertain_nodes} \leftarrow V \setminus \{s, t\} \quad (15)$$

Main Loop:

$$\textbf{while } \text{uncertain_nodes} \neq \emptyset : \quad (16)$$

$$\quad \text{improved} \leftarrow \text{false} \quad (17)$$

$$\quad v_{\text{worst}} \leftarrow \arg \max_{v \in \pi} \sigma_{\min}(v \mid \text{weights_dict}) \quad (18)$$

$$\quad \text{for each } v \in \pi \setminus R : \quad (19)$$

$$\quad \quad // \text{Synthesize weights for } v\text{'s neighbors} \quad (20)$$

$$\quad \quad \text{neighbors} \leftarrow \{u : (v, u) \in E \text{ or } (u, v) \in E\} \quad (21)$$

$$\quad \quad \text{for each } u \in \text{neighbors} : \quad (22)$$

$$\quad \quad \quad \text{if } (v, u) \notin \text{weights_dict} : \quad (23)$$

$$\quad \quad \quad \quad w(v, u) \leftarrow \text{synthesize}(v, u, \beta_0) \quad (24)$$

$$\quad \quad \quad \quad \text{weights_dict}[(v, u)] \leftarrow w(v, u) \quad (25)$$

$$\quad \quad \Sigma(v) \leftarrow \text{fuzzy_separation_cost}(v, \text{weights_dict}) \quad (26)$$

$$\quad \quad // \text{Check deterministic separation} \quad (27)$$

$$\quad \quad \text{if } \Sigma(v) \text{ and } \Sigma(v_{\text{worst}}) \text{ are } \beta_0\text{-separated} : \quad (28)$$

$$\quad \quad \quad \pi \leftarrow \text{improve_path}(v, \text{weights_dict}) \quad (29)$$

$$\quad \quad \quad R \leftarrow R \cup \{v\} \quad (30)$$

$$\quad \quad \quad \text{improved} \leftarrow \text{true} \quad (31)$$

$$\quad \quad \quad \text{break} \quad (32)$$

$$\quad \text{if not improved:} \quad 14 \quad (33)$$

$$\quad \quad \text{uncertain_nodes} \leftarrow \text{uncertain_nodes} \setminus (\text{nodes evaluated}) \quad (34)$$

$$\quad \quad \text{if uncertain_nodes} = \emptyset : \quad (35)$$